

Format

格式手册

在文章开始之前，**OI Wiki** 项目组全体成员十分欢迎您为本项目贡献页面。正因为有了上百位像您一样的人，才有了 **OI Wiki** 的今天！

本页面将列出在 **OI Wiki** 编写过程时推荐使用的格式规范与编辑方针。请您在撰稿或者修正 Wiki 页面以前，仔细阅读以下内容，以帮助您完成更高质量的内容。

如果您已迫不及待，想要快速上手，建议先阅读图片举例的章节。

贡献文档要求

当你打算贡献某部分的内容时，你应该尽量熟悉以下三部分：

- 文档存储的格式
- 文档的合理性
- remark-lint 和公式的格式要求

文档引用与存储的格式

- 文件名请务必都小写，以 - 分割。例如：`file-name.md`。
- 请务必确保文档中引用的 **外链** 图片已经全部转存到了 **本库内** 对应的 `images` 文件夹中（防止触发某些网站的防盗链），建议处理成 MD 文档名称 + 编号 的形式（可参考已有文档中图片的处理方式）。例如：本篇文档的文件名称为 `format`，则文档中引用的第一张图片的名字为 `format1.png`。
- 推荐使用 SVG 格式的图片⁶，以获取较好的清晰度和缩放效果。
- 动图如果无法或者不会制作 SVG 格式的，则推荐使用 APNG 格式⁸的文件。Windows 用户可使用 [ScreenToGif](#) 录制，Linux 用户可使用 [Peek](#) 录制，注意需要在设置里调整为录制 APNG。其他情况则推荐先制作为 MP4 等视频文件再转换为 APNG，如果使用 `ffmpeg` 则可以使用 `ffmpeg -i filename.mp4 -f apng filename.apng -plays 0` 转换。⁹
- 同时具有源文件和导出图像的图片（例如 JPG 文件与 PSD 文件或者 SVG 图像与 TikZ TeX 源代码），建议将源文件以与图片相同的文件名保存于同一目录下。
- 请确保您的文档中的引用链接的稳定性。**不推荐** 引用 **自建** 服务中的资源（如自建 OJ 里的题目）。建议在添加时同时将该外链存于互联网档案馆⁷，以防无法替代的链接失效。
- 站内链接请去掉网站域名，并且使用相对路径链接对应 `.md` 文件。例如，在本页面 (`intro/format`) 中链接杂项简介 (`misc`)，应使用 `[杂项简介]` (`../misc/index.md`)。可以在链接中添加 hash 来链接到某一节，例如 `[Pull Request 信息格式规范](./htc.md#pull-request-信息格式规范)`，hash 的值可以通过位于每个标题右侧的按钮或者位于网页右侧的目录中的链接得到。

文档的合理性

合理性，指所编写的内容必须具有如下的特性：

- 由浅入深，内容的难度应该具有渐进性。
- 逻辑性。
 - 对于算法或数学概念类内容的撰写应该尽量包含以下的内容：
 1. 原理：说明该内容对应的原理；
 2. 例子：给出 1 ~ 2 个典型的例子；
 3. 题目：在该标题下，只需要给出**题目名字和题目链接**。对于算法类题目，题目链接 OJ 的优先级为：原 OJ（国外 OJ 要求国内可流畅访问）> UOJ > LOJ > 洛谷。
 - 示例页面：[IDA*](#)
 - 对于工具类内容的撰写应该尽量包含以下的内容：
 1. 简介：阐明该工具的背景与用途。
 2. 配置方式：详细给出配置环境与使用的过程，下载与安装方法建议尽量引用官方文档。
 - 示例页面：[WSL \(Windows 10\)](#)

除现有内容质量较低的情况外，建议尽量从 **补充** 的角度来做贡献，而非采取直接覆盖的方式。如果拿不准主意，可以参考 [关于本项目的交流方式](#) 一节，与 **OI Wiki** 项目组联系。

文档的基本格式要求

Remark-lint 的格式要求

[remark-lint](#) 可以自动给项目内文件统一风格。**OI Wiki** 现在启用的配置文件托管在 [.remarkrc](#)。

在配置过程中 **OI Wiki** 项目组也遇到了一些 remark-lint 不能很好处理的问题，所以请严格按照下列要求编辑文档：

- 不要使用如 `<LTh1>` 或者 `# 标题` 的一级标题。
- 标题要空一个英文半角空格，例如：`# 简介`。

- 由于 remark-lint 不能很好地处理删除线，因此请不要使用删除线语法（不使用删除线语法的另外一个原因是，删除线划去的内容大多为「抖机灵」性质，对读者理解帮助不大，不符合下面的「文本内容的格式要求」中对内容表述的要求）。
- 列表：
 - 列表前要有空行，新开一段。
 - 使用有序列表（如 1. 例子）时，点号后要有空格。
- 行间公式前后各要有一行空行，否则会被当做是行内公式。
- 使用 ??? 或 !!! 开头的 Details 语法时，每一行要包括在 Details 语法的文本框的文本，开头必须至少有 4 个空格。

即使是空行，也必须保持与其他行一致的缩进。请不要使用编辑器的自动裁剪行末空格功能。

▼ 示例

```
1 ???+ warning
2     请记得在文本前面添加 4 个空格。其他的语法还是与 Markdown 语法一致。
3
4     不添加 4 个空格的话，文本就不会出现在 Details 文本框里了。
5
6     这个`???`是什么的问题会在下文解答。
```

▼ Warning

请记得在文本前面添加 4 个空格。其他的语法还是与 Markdown 语法一致。

不添加 4 个空格的话，文本就不会出现在 Details 文本框里了。

这个 ??? 是什么的问题会在下文解答。

- 代码样式的纯文本块请使用 ```text`。直接使用 ````` 而不指定纯文本块里的语言，可能会导致内容被错误地缩进。

标点符号的使用

- 请在每句话的末尾添加 句号。
- 请正确使用 全角 标点符号与 半角 标点符号。汉语请使用全角符号，英语请使用半角符号。中文中夹用英文时，请参考 [中文出版物夹用英文的编辑规范](#)。
- 由于 “” 未区分全半角，请使用 「」 作为全角引号， “.” 作为半角引号。
- 注意区分 顿号 与 逗号 的使用。
- 注意 括号 的位置。句内括号与句外括号的位置不同。
- 通常使用 分号 来表示列表环境中各复句之间的关系。
- 对于有序列表，推荐在每一项的后面添加 分号，在列表最后一项的后面添加 句号；对于无序列表，推荐在每一项的后面添加 句号。
- 注意区分各种不同的连接号，如 hyphen（一般使用 U+002D hyphen-minus (-)，即键盘上的「减号」代替），U+2013 en dash (—) 和 U+2014 em dash (——)。（英文中连接多个人名时，须用 en dash，但是极常见误用为 hyphen。其他误用较为罕见，基本上只需记住这一点即可。）详见 [连接号 - 维基百科](#)。

▼ 示例

- 中学生学科竞赛主要包括信息学奥林匹克竞赛、信息学奥林匹克竞赛、信息学奥林匹克竞赛、信息学奥林匹克竞赛和信息学奥林匹克竞赛（谁写的这个示例，建议抬走）。
- 「你吃了吗？」李四问张三。
- 我想对你说：「我真是太喜欢你了。」
- 「苟利国家生死以，岂因祸福避趋之！」
- 张华考上了大学；李萍进了技校；我当了工人：我们都有美好的前途。¹
- 以下是这个算法的基本流程：
 1. 初始化到各点的距离为无穷大，将所有点设置为未被访问过，初始化一个队列；
 2. 将起点放入队列，将起点设置为已被访问过，更新到起点的距离为；
 3. 取出队首元素，将该元素设置为未被访问过；
 4. 遍历所有与此元素相连的边，若到这个点存在更短的距离，则进行松弛操作；
 5. 若这个点未被访问过，则将这个点放入队列，且设置这个点为已经访问过；
 6. 回到第三步，直到队列为空。
- KMP 算法（Knuth–Morris–Pratt algorithm, KMP algorithm）由 Knuth、Pratt 和 Morris 在 1977 年共同发布。²

Markdown 格式与主题扩展格式要求

- 表示强调时请使用 **SOMETHING** 和 「」，而非某级标题，因为使用标题会导致文章结构层次混乱和（或）目录出现问题。
- 当需要引用题目链接时，应尽可能使用原 OJ 题库中的链接而不是镜像链接。
- 请正确使用 Markdown 的区块功能。插入行内代码请使用一对反引号包围代码区块；行间代码请使用一对 ````` 包围代码区块，其中反引号就是键盘左上角波浪线下面那个符号，行间代码请在第一个 ````` 的后面加上语言名称（如：````cpp`）。

▼ 示例

```
1 ```cpp
2 // #include<LStdio.h>    //不好的写法
3 #include <LStdio>    //好的写法
4 ```

1 // #include<LStdio.h>    //不好的写法
2 #include <LStdio>    //好的写法
```

- 「参考资料与注释」使用 Markdown 的脚注功能进行编写。格式为：

- 1 文本内容。[^脚注名]
- 2 [^脚注名]: 参考资料内容。注意: 冒号是英文冒号, 冒号后面跟着一个空格。

脚注名既可以使用数字也可以使用文本。脚注名摆放的位置与括号的用法一致。为美观起见, 建议同一个页面内的脚注名遵循统一的命名规律, 如: ref1、ref2、note1.....

脚注的内容统一放在 ## 参考资料与注释 二级标题下。

▼ 示例

```
1 当 `#include <Ltcxxx>` 可以替代 `#include <Ltxxxx.h>` 时, 应使用前者。[^ref1]
2
3 2020年1月21日, CCF宣布恢复NOIP。[^ref2]
4
5 ## 参考资料与注释
6
7 [^ref1]: [cstdio stdio.h namespace](https://stackoverflow.com/questions/10460250/cstdio-stdio-h-namespace)
8
9 [^ref2]: [CCF关于恢复NOIP竞赛的公告-中国计算机学会](https://www.ccf.org.cn/c/2020-01-21/694716.shtml)
```

当 #include <Ltcxxx> 可以替代 #include <Ltxxxx.h> 时, 应使用前者。³

2020 年 1 月 21 日, CCF 宣布恢复 NOIP。⁴

- 建议使用主题扩展的 ???+note 格式 (即 [Collapsible Blocks](#)) 来描述题面和参考代码。也可以用这种格式来展示其他需要补充介绍的内容。

示例代码:

```
1 ??? note "标题"
2     这个文本框会被默认折叠。
3
4     推荐将 **解题代码** 放在折叠文本框内。
5
6 ???+note "[HDOJ 的「A + B Problem」](https://acm.hdu.edu.cn/showproblem.php?pid=1000)"
7     标题也可以使用 Markdown 的超链接。这里的超链接是 HDOJ 的「A + B Problem」 。
8
9     而且推荐以这种方式**标注原题链接**。
10
11     注意双引号的位置。
```

效果:

► 标题

▼ HDOJ 的「A + B Problem」

标题也可以使用 Markdown 的超链接。这里的超链接是 HDOJ 的「A + B Problem」。

而且推荐以这种方式 标注原题链接。

注意双引号的位置。

两种格式的区别是, 带 + 的会默认保持展开, 而不带 + 的会默认保持折叠。

折叠框的标题, 即 ???+note 中 note 后的内容应以 " 包裹起来。其中的内容支持 Markdown 语法。详见 [Admonition - Changing the title](#)。(不具备折叠功能的为一般的 Admonitions, 参考 [Admonitions - Material for MkDocs](#))

- 当需要添加不同语言的代码时, 推荐使用 Content tabs, 可以实现不同语言代码的切换。Content tabs 还有其他的用法, 详见 [Content tabs](#)。其使用方法和效果如下。

▼ 示例

```
1 注意需要在文本前面添加 4 个空格。其他的语法还是与 Markdown 语法一致。
2
3 === "C"
4
5     ``` c
6     #include <Ltstdio.h>
7
8     int main(void) {
9         printf("Hello world!\n");
10        return 0;
11    }
12    ```
13
14 === "C++"
15
16     ``` c++
17     #include <Ltiostream>
18
19     int main(void) {
20         std::cout << "Hello world!" << std::endl;
21         return 0;
22     }
23     ```
```

CC++

```
1 #include <Tstdio.h>
2
3 int main(void) {
4     printf("Hello world!\n");
5     return 0;
6 }

1 #include <Tiostream>
2
3 int main(void) {
4     std::cout << "Hello world!" << std::endl;
5     return 0;
6 }
```

如果对 mkdocs-material（我们使用的这个主题）还有什么问题，还可以查阅 [MkDocs 使用说明](#)，其介绍了 mkdocs-material 主题的插件使用方式。

文本内容的格式要求

- 所有的 **OI Wiki** 文本都应使用粗体标记。
- 在页面的开头应有一段简短的文字（如「本页面将介绍……」），用于概述页面内容。

▼ 示例

本页面将列出在 **OI Wiki** 编写过程时推荐使用的格式规范与编辑方针。

- 涉及到「前置知识」的页面，请在开头添加一行 **前置知识**：……，放在页面概述前。格式如下：

前置知识：[站内页面1] (url1)、[站内页面2] (url2) 和 [站内页面3] (url3)

▼ 示例

前置知识： [时间复杂度](#)

本页面将介绍基础的计算理论的知识。

- 请注意文档结构。文档结构应当十分条理，层次清晰。请不要让诸如「五级标题」这种事情再次发生了，一篇正常的文章是用不到如此复杂的结构层次的。
- 请注意内容的表述。作为一个百科网站，**OI Wiki** 使用的语言应该是书面的，客观的。诸如「抖机灵」性质的，对读者理解帮助不大的内容，不应该出现在 **OI Wiki** 当中。
- 请尽量为链接提供完整的标题、或者可被识别的提示，避免使用裸地址和「这」、「此」之类的模糊不清的描述。每一个超链接都应尽量对其加以清楚明确的描述，方便读者明白该超链接将指向何处。

建议使用源文章或者标签页的标题。

▼ 不推荐的写法

```
1 请参考 [这个页面] (https://docs.github.com/en/github/collaborating-with-issues-and-pull-requests/syncing-a-fork)
2
3 请参考 &LThttps://docs.github.com/en/github/collaborating-with-issues-and-pull-requests/syncing-a-fork>
```

请参考 [这个页面](#)

请参考 <https://docs.github.com/en/github/collaborating-with-issues-and-pull-requests/syncing-a-fork>

▼ 推荐的写法

```
1 请参考 GitHub 官方的帮助页面 [Syncing a fork - GitHub Docs] (https://docs.github.com/en/github/collaborating-with-issues-and-pull-requests/syncing-a-fork)
```

请参考 GitHub 官方的帮助页面 [Syncing a fork - GitHub Docs](#)

- 受 Markdown 格式限制，## 参考资料与注释 二级标题必须放在文末。
- 所有用作序号的数字建议使用中文。示例：
 - 数列的第一项。
 - 输入文件的第一行。
- 请尽量避免在标题中使用 MathJax 公式，无论是几级标题。在标题中使用公式有可能会致目录显示错误。⁵
- 请注意代码的可读性。
 - 代码应拥有清晰的逻辑。
 - 建议在参考代码中添加适当注释以方便读者理解。
 - 尽量避免出现影响阅读的预编译指令和宏定义。

LaTeX 公式的格式要求

LaTeX 作为公式排版的首选，我们应当正确地使用它。因此对于 LaTeX 的使用我们有严格的要求。如果您想要快速上手，可以阅读本章节末给出的表格。

- 您使用的符号不应与 [数学符号表](#) 规定的符号冲突。

- 使用 Roman 体表示数字、常量、算子和函数。使用 Italic 体表示变量、下标。LaTeX 已经预先定义好了一些常见的常量、函数、运算符等，我们可以直接调用，包括但不限于：

```
1 \log, \ln, \lg, \sin, \cos, \tan, \sec, \csc, \cot, \gcd, \min, \max, \exp, \inf, \mod, \bmod, \pmod
```

所以在输入常量、函数名、运算符等时，请先检查一下是否应该使用 Roman 体或其它字体。LaTeX 符号的书写可参考 [KaTeX 的 Supported Functions 页面](#)（不是全部），也可以搜索求解。

由于 LaTeX 书写 Roman 体小写希腊字母较为困难，故小写希腊字母常量、算子和函数可以使用 Italic 体，如 $\ln$ 以及 $\lg$ 中的 $\ln$ 。

如果遇到没有预先定义好的需要使用 Roman 体的 **函数名**，我们可以使用 $\operatorname{something}$ 来产生，如我们可以使用 lcm 产生正体的最小公倍数（函数）符号。同理，产生 Roman 体的 **常量** 应用 $\mathrm{}$ ；产生 Roman 体粗体符号应用 $\mathbf{}$ ；产生 Italic 体粗体符号应用 $\boldsymbol{}$ （如向量 $\mathbf{i}$ ）。对于多字母的变量，应当使用 $\textit{}$ 。其他非数学内容，包括英文、特殊符号等，一律使用 $\text{}$ 。中文我们则建议不放在 LaTeX 公式中。

- 如果表达式须折行（常见于较长的行间公式中），则应遵循如下换行规则：
 - 将换行符放在 $+$ ， $-$ ， $\times$ ， $\div$ 之前，如果有必要，也可放在 $=$ 之前，如：
$$a + b + c + d + e + f + g + h + i + j + k + l + m + n + o + p + q + r + s + t + u + v + w + x + y + z + \dots$$
 - 同一运算符不应在换行符前后同时出现，
 - 换行符尽量不要出现在括号内的表达式中。
- 在行内使用分数的时候，请使用 $\dfrac{ }{ }$ 。比如 $\dfrac{1}{2}$ ，效果 $\frac{1}{2}$ ，而不是 $\frac{1}{2}$ ，效果 $\frac{1}{2}$ 。
- 组合数请使用 $\dbinom{n}{m}$ ，效果 $\binom{n}{m}$ ，而不是 $\{n \choose m$ （在 LaTeX 中这种写法已不推荐）；与上一条关于分数的约定相似，请不要使用 $\binom{n}{m}$ ，效果 $\binom{n}{m}$ 。
- 尽可能避免在行内使用巨运算符（如 $\sum$ ， $\prod$ 等）。
- 在不会引起歧义的情况下，请用 $\times$ 代替星号，叉乘请使用 $\times$ ，点乘请使用 $\cdot$ 。如 $a \times b$ ，而不是 $a * b$ 。
- 请用 $\cdots$ （居于排版基线与顶线中间）， $\ldots$ （居于排版基线的位置）， $\vdots$ （竖着的省略号）代替 $...$ 。如 $a \cdots b$ ，而不是 $a \dots b$ 。
- 请注意，不要在非代码区域使用任何程序设计语言的表示方式，而是使用 LaTeX 公式。例如，使用 $=$ 而不是 $==$ （如 $a = b$ ，而不是 $a == b$ ）、使用 $\leq$ 或者 $\leq$ 2 而不是 $a < \leq b$ 、使用 $a \bmod b$ 代替 $a \% b$ （如 $a \bmod b$ ，而不是 $a \% b$ ）等。
- 公式中不要使用中括号连缀（即 C++ 高维数组的表示方式）而多使用下标。即 $a[i]$ 而不是 $a[i]$ 。在公式中下标较复杂的情况下建议改用多元函数 $f(i, j, k)$ 或内联代码格式。对于一元简单函数使用 f_i 、 $f(i)$ 或 $f[i]$ 均可。
- 为了统一且书写方便，复杂度分析时大 O 记号请直接使用 $O()$ 而不是 $\mathcal{O}()$ 。
- 在表示等价关系时，请使用 $\iff$ ，效果 $\iff$ ，而不是 $\Leftrightarrow$ ，效果 $\Leftrightarrow$ 。
- 分段函数环境 `cases` 只能有两列（即一个 $\&$ 分隔符）。
- 请不要滥用 LaTeX 公式。这不仅会造成页面加载缓慢（因为 MathJax 的效率低是出了名的），同时也会导致页面的排版混乱。我们通常使用 LaTeX 公式字体表示变量名称。我们的建议是，如非必要，尽量减少公式与普通正文字体的大量混合使用，如非必要，尽量不要使用公式，如：

```
1 我们将要学习  $\textit{Network-flow}$  中的  $\textit{SPFA}$  最小费用流，需要使用  $\textit{Edmonds-Karp}$  算法进行增广。
```

就是一个典型的 **滥用公式字体** 的例子。（在页面中使用斜体请用 $\textit{文本}$ 表示。）

- 请正确使用对应的 LaTeX 符号，尤其是公式中的希腊字母等特殊符号。如欧拉函数请使用 φ ，圆的直径请使用 Φ ，黄金分割请使用 ϕ 。这些符号虽然同样表示希腊字母 Phi，但是在不同的环境下有不同的含义。切记 **不要使用输入法的插入特殊符号** 来插入这种符号。

另外，由于 LaTeX 历史原因，空集的符号应为 $\varnothing$ 而不是 $\emptyset$ ；其他的符号应参照 [数学符号表](#) 书写。

我们可以使用一个表格来总结一下上述内容。注意本表格没有举出所有符号的用法，只给出常见的错误。类似的情况类比即可。

不符合规定的用法	渲染效果	符合规定的用法	渲染效果
$\log$, $\ln$, $\lg$	$\log$, $\ln$, $\lg$	$\log$, $\ln$, $\lg$	$\log$, $\ln$, $\lg$
$\sin$, $\cos$, $\tan$	$\sin$, $\cos$, $\tan$	$\sin$, $\cos$, $\tan$	$\sin$, $\cos$, $\tan$
$\gcd$, lcm	$\gcd$, lcm	$\gcd$, lcm	$\gcd$, lcm
e , $\textit{e}$ （自然对数的底）	e , $\textit{e}$	e	e
i , $\textit{i}$ （虚数单位）	i , $\textit{i}$	i	i
$\text{小于 } a \text{ 的质数}$	$\text{小于 } a \text{ 的质数}$	$\text{小于 } a \text{ 的质数}$	$\text{小于 } a \text{ 的质数}$
$\dots$	$\dots$	$\cdots$, $\ldots$, $\vdots$, $\ddots$	$\cdots$, $\ldots$, $\vdots$, $\ddots$
$a * b$ （两个数相乘）	$a * b$	$a \times b$, $a \cdot b$	$a \times b$, $a \cdot b$
$\textit{SPFA}$ （英文名称）	$\textit{SPFA}$	$\textit{SPFA}$	$\textit{SPFA}$
$a = b$	$a = b$	$a = b$	$a = b$
$f[i][j][k]$	$f[i][j][k]$	$f_{i,j,k}$, $f(i,j,k)$	$f_{i,j,k}$, $f(i,j,k)$
R, N^* （集合）	R, N^*	$\mathbf{R}$, $\mathbf{N}^*$	$\mathbf{R}$, $\mathbf{N}^*$
$\emptyset$	$\emptyset$	$\varnothing$	$\varnothing$
$\textit{size}$	$\textit{size}$	$\textit{size}$	$\textit{size}$

对数学公式的附加格式要求

请注意，尽管上述输入公式的语法和真正的 LaTeX 排版系统非常相似，但 **MathJax** 和 **LaTeX** 是两个完全没有关系的东西，MathJax 仅仅使用了一部分与 LaTeX 非常相似的语法而已。实际上，二者之间有不少细节差别，而这些差别经常导致写出来的公式在二者之间不通用。

由于 **OI Wiki** 使用 LaTeX 排版引擎开发了 PDF 导出工具，因此有必要强调公式在 MathJax 和 LaTeX 之间的兼容性。请各位在 **Wiki** 中书写数学公式时注意以下几点。

这些规则已经向 MathJax 做了尽可能多的妥协。导出工具兼容了一部分原本仅能在 MathJax 中正常输出的写法。

- 请使用 `\begin{aligned} ... \end{aligned}` 表示多行对齐的公式；
- 如果这些多行对齐的公式需要 编号，请用 `align` 或 `equation` 环境；
- 不要使用 `split`、`eqnarray` 环境；
- 不要使用 `\lt`、`\gt` 来表示大于号和小于号，请直接使用 `<`、`>`；
- 不要直接用 `\\` 换行（需要换行的公式，请套在 `aligned` 或其他多行环境下）；
- 若要输出 LaTeX 符号 请用 `\rm{\LaTeX}`，而不是 `\mathrm`；（LaTeX 在 TeX 排版系统中是一个不能用于数学模式下的命令，而 `\mathrm` 又不能在普通模式下使用；另外，`\text` 命令虽然在 TeX 上正常输出，但是在 MathJax 中 `\text` 命令的参数会被原样输出，而不是按命令转义）；
- 数学公式中的中文文字 必须置于 `\text{}` 命令之中，而变量、数字、运算符、函数名称则必须置于 `\text{}` 命令之外。请不要在 `\text{}` 命令中嵌套数学公式；
- 使用 `array` 环境时请注意 实际列数与对齐符号的数量保持一致。例如下面的公式中，数据实际有 3 列（`&` 是列分隔符），因此需要 3 个对齐符号（`l/r/c` 分别表示左、右、居中对齐）。

```
1 $$
2 \begin{array}{lll}
3 F_1=\{\frac{0}{1},\&\frac{1}{1}\}\backslash\backslash
4 F_2=\{\frac{0}{1},\&\frac{1}{2},\&\frac{1}{1}\}\backslash\backslash
5 \end{array}
6 $$
```

伪代码格式

伪码具体格式没有严格要求，请参考算法导论或学术论文。注意不要写成 Python。

Wiki 内使用 LaTeX 书写伪码，整体处于 `array` 环境中，缩进使用 `\qqquad`，文字描述使用 `\text`，关键字使用 `\textbf`，赋值使用 `\gets`。

参考示例：

```
1 $$
2 \begin{array}{l}
3 1 \& \textbf{Input. } \text{The edges of the graph } e , \text{ where each element in } e \text{ is } (u, v, w) \backslash\backslash
4 \& \text{ denoting that there is an edge between } u \text{ and } v \text{ weighted } w . \backslash\backslash
5 2 \& \textbf{Output. } \text{The edges of the MST of the input graph}.\backslash\backslash
6 3 \& \textbf{Method. } \backslash\backslash
7 4 \& \text{result} \gets \text{varnothing} \backslash\backslash
8 5 \& \text{sort } e \text{ into nondecreasing order by weight } w \backslash\backslash
9 6 \& \textbf{for} \text{ each } (u, v, w) \text{ in the sorted } e \backslash\backslash
10 7 \& \quad \textbf{if } u \text{ and } v \text{ are not connected in the union-find set } \backslash\backslash
11 8 \& \quad \quad \text{connect } u \text{ and } v \text{ in the union-find set} \backslash\backslash
12 9 \& \quad \quad \text{result} \gets \text{result} \cup (u, v, w) \backslash\backslash
13 10 \& \textbf{return } \text{result}
14 \end{array}
15 $$
```

代码块的格式要求

代码块目前分为两种：片段和例题。

关于片段代码：

- 片段的代码内容请直接在 Markdown 文档中修改。

关于例题代码：

- 例题代码的表示形式为 `--8<-- "path"`，代码均存储在 `path` 中。路径通常为 `docs/主题/code/内容/内容_编号.cpp`。
- 修改例题代码时，请保证你的代码是正确的。例题代码均拥有一组测试数据，存储在 `//docs/主题/examples/内容/内容_编号.in/ans` 中。

如果你需要添加例题：

- 请在 `docs/主题/code/内容` 中添加你的例题代码，并编号。通常，该 `内容` 文件夹中已经有了一个或者多个代码。例子：如果需要修改 `dag.md` 的代码，那么路径为 `docs/dp/code/dag`，其中 `dp` 为主题，而 `dag` 为内容。
- 如果需要在所有例题的最后添加一个例题代码，请顺延目前的编号。比如已经存在了 `code/prefix-sum/prefix-sum_3.cpp`，如果需要在最后一个例题后继续添加一个例题，请将你的代码命名为 `prefix-sum_4.cpp` 并添加到 `docs/basic/code/prefix-sum` 中。
- 如果需要在文章中间添加一个例题代码，请插入并改变原先的编号。比如已经存在了 `prefix-sum_2.cpp` 和 `prefix-sum_3.cpp`，如果你需要在第二个例题和第三个例题中间再添加一个例题，请将你的代码命名为 `prefix-sum_3.cpp` 并将原先的 `prefix-sum_3.cpp` 改名为 `prefix-sum_4.cpp` 同时在 **Markdown** 文档和测试数据存放的

文件夹中同步修改编号。

- 别忘记，你还要对你的代码添加一组测试数据，以保证这个代码是可以成功运行的。你需要在 docs/主题/examples/内容 文件夹中添加一组测试数据，将输入数据存储为 内容_编号.in，将标准答案存储为 内容_编号.ans。
- 最后，可以将代码添加到文档中了。请直接 在文档中用添加代码块的格式，并将代码块内部直接写成 `--8<--` “你的代码路径” 的格式就可以了。

OI Wiki 会对例题代码进行全平台测试，为保证您的代码能够顺利通过测试，请遵守如下规则：

- 您的代码需要同时支持在 C++14、C++17、C++20 标准下编译和运行。
- 不要使用 `<bits/stdc++.h>`、`<extc++.h>` 等非标准头文件。
- 标准答案文件不要有多余空格。
- 不要使用 [代用记号](#)。
- 使用 [聚合初始化](#) 时，`object{args}` 不可写成 `(object){args}`。
- 使用 [运算符重载](#) 时注意格式，如重载比较运算符时，若使用成员函数写法，则不可省略 `const` 限定符。
- 不要使用类似 `#define int long long` 的宏定义。
- 若您需要使用 C 风格的 [有格式输入/输出](#)，请特别留意格式指示符的写法：如 `size_t` 对应 `%zu`，`ptrdiff_t` 对应 `%td`。例如输出某 STL 容器的大小时，代码应类似 `printf("%zu", container.size());`。
- 由于当前测试环境 `libstdc++` 的 `<chrono>` 库有 [BUG](#)，所以请避免使用 `<chrono>` 库。
- 由于 `long` 与 `unsigned long` 在某些测试环境下为 32 位，而在另一些测试环境下为 64 位，为确保各平台代码行为一致，故不推荐使用这两种类型。推荐使用 [定宽整数类型](#)。
- 不建议使用 `__gcd`、`__int128`、`__builtin_` 系列函数等非标准内容。如果您需要使用，则需确保您的代码能通过全平台测试，如 [此代码](#) 提供了 `libstdc++` 中 `std::bitset` 特有成员函数 `_Find_first()` 的全平台实现。

此外，为了提高代码的可读性，建议遵守如下规则：

- 代码应尽可能简洁易懂，不要过度压行，不要引入过多无关代码（如未使用的宏定义等）。
- 不推荐对函数使用 `inline` 关键字，详见 [编译优化](#)。
- 不要用 `0` 代替 `false/NULL/nullptr` 等，不要用 `1` 代替 `true` 等。
- 在声明 [类型别名](#) 时，不推荐使用 `typedef`，推荐使用 `using`。
- 不推荐用宏定义定义常量，推荐直接使用 `constexpr/const` 等关键字定义常量。

图解

可能上述要求把握起来有些困难，接下来我们给出一些图片来具体分析哪种格式应该使用，哪种不该使用：

例 1

Description

给定多项式 $f(x), g(x)$ ，求 $g(x)$ 除 $f(x)$ 的商 $Q(x)$ 和余数 $R(x)$ 。

Method

发现若能消除 $R(x)$ 的影响则可直接 [多项式求逆](#) 解决。

考虑构造变换

$$f^R(x) = x^{\deg f} f\left(\frac{1}{x}\right)$$

观察可知其实质为反转 $f(x)$ 的系数。

设 $n = \deg f, m = \deg g$ 。

将 $f(x) = Q(x)g(x) + R(x)$ 中的 x 替换成 $\frac{1}{x}$ 并将其两边都乘上 x^n ，得到：

$$\begin{aligned} f\left(\frac{1}{x}\right) &= x^{n-m} Q(x) x^m g(x) + x^{n-m+1} x^{m-1} R(x) \\ f^R(x) &= Q^R(x) g^R(x) + x^{n-m+1} R^R(x) \end{aligned}$$

注意到上式中 $R^R(x)$ 的系数为 x^{n-m+1} ，则将其放到模 x^{n-m+1} 意义下即可消除 $R^R(x)$ 带来的影响。

又因 $Q^R(x)$ 的次数为 $(n-m) < (n-m+1)$ ，故 $Q^R(x)$ 不会受到影响。

则：

将复杂的 LaTeX 公式使用行间格式，可以使得页面错落有致。但 **OI Wiki** 作为一个以中文为主体的站点，我们希望大部分纲领性的信息（如标题）尽量使用中文（除英文专有名词）。

例 2

复杂度分析

对于 $F_k(n)$ 的计算，其第一种方法的时间复杂度被证明为 $O(n^{1-\epsilon})$ （见 zzt 集训队论文 2.3）；

对于第二种方法，其本质即为洲阁筛的第二部分，在洲阁论文中也有提及（6.5.4），其时间复杂度被证明为 $O\left(\frac{n^{\frac{3}{4}}}{\log n}\right)$ 。

对于 $F_{\text{prime}}(n)$ 的计算，事实上，其实现与洲阁筛第一部分是相同的。

考虑对于每个 $m = n/i$ ，只有在枚举满足 $p_k^2 \leq m$ 的 p_k 转移时会对时间复杂度产生贡献，则时间复杂度可估计为：

$$\begin{aligned} T(n) &= \sum_{i^2 \leq n} O(\pi(\sqrt{i})) + \sum_{i^2 \leq n} O\left(\pi\left(\sqrt{\frac{n}{i}}\right)\right) \\ &= \sum_{i^2 \leq n} O\left(\frac{\sqrt{i}}{\ln \sqrt{i}}\right) + \sum_{i^2 \leq n} O\left(\frac{\sqrt{\frac{n}{i}}}{\ln \sqrt{\frac{n}{i}}}\right) \\ &= O\left(\int_1^{\sqrt{n}} \frac{\sqrt{\frac{n}{x}}}{\log \sqrt{\frac{n}{x}}} dx\right) \\ &= O\left(\frac{n^{\frac{3}{4}}}{\log n}\right) \end{aligned}$$

对于空间复杂度，可以发现不论是 F_k 还是 F_{prime} ，其均只在 n/i 处取有效点值， $\clubsuit O(\sqrt{n})$ 个。

则可以使用杜教筛一节中介绍的 [trick](#) 来将空间复杂度优化至 $O(\sqrt{n})$ 。

较复杂度的 LaTeX 公式请注意等号的对齐，同时可以适当引用 Wiki 的页面 [链接](#) 来完善内容。

例 3

本题中用来实现语音识别，其实就是找一条对应的概率最大的路径。

ref: <https://segmentfault.com/a/1190000008720143>

基于接缝裁剪的图像压缩

玩过 opencv 的应该有印象，seam carving 就是在做 dp。

题中要求每一行删除一个像，每个像素都有代价，要求总代价最小。

限制：要求相邻两行中删除的像素必须位于同一列或相邻列。

$$dp[i][j] = \min(dp[i-1][j], dp[i-1][j-1], dp[i-1][j+1]) + cost[i][j]$$

边界： $dp[1][i] = cost[1][i]$ 。

字符串拆分

相当于问怎么按顺序拼起来使得总代价最小。

等价于之前那个最优二叉搜索树。

$$dp[i][j] = \min(dp[i][k] + dp[k][j]) + l[j] - l[i] + 1, k = i + 1 \cdots j - 1$$

注意 $l[i]$ 表示的是第 i 个切分点的位置。

边界： $dp[i][i] = 0$ 。

就按照区间 dp 的姿势来写就好了。



机次策略规划

一般情况下，我们建议将引用的资料列在文末的 ##参考资料与注释 一节，并在原句后面加上脚注，而不是直接给出链接。同时一定要避免使用 LaTeX 公式表达代码，上图中两个中括号就是不规范的写法。我们建议使用 `dp(i, j)` 或者 `dp_{i,j}`。

例 4

矩阵链乘法

给出 n 个矩阵的序列，希望计算他们的乘积，问最少需要多少次乘法运算？

(认为 $p \times q$ 的矩阵与 $q \times r$ 的矩阵相乘代价是 $p \times q \times r$ 。)

完全括号化方案是指要给出谁先和谁乘。

动态规划原理

两个要素：

最优子结构

具有最优子结构也可能是适合用贪心的方法求解。

注意要确保我们考察了最优解中用到的所有子问题。

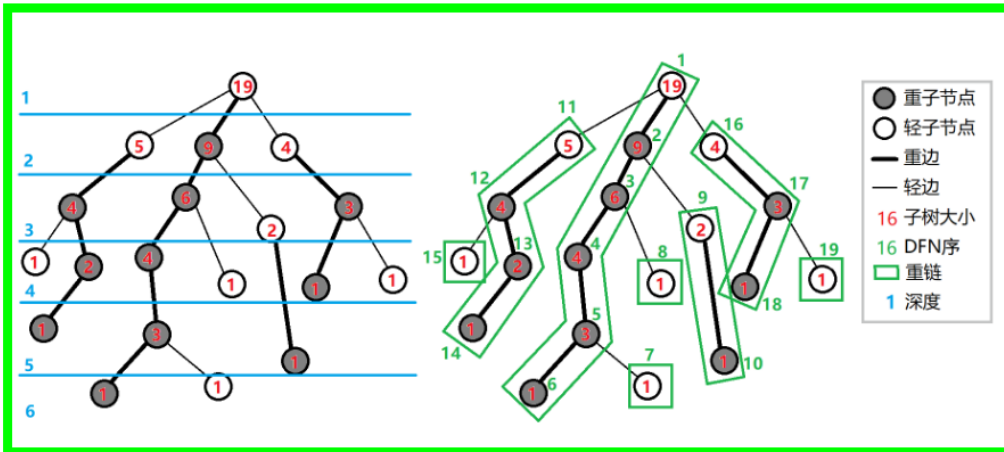
1. 证明问题最优解的第一个组成部分是做出一个选择；
2. 对于一个给定问题，在其可能的第一步选择中，你界定已经知道哪种选择才会得到最优解。你现在并不关心这种选择具体是如何得到的，只是假定已经知道了这种选择；
3. 给定可获得的最优解的选择后，确定这次选择会产生哪些子问题，以及如何最好地刻画子问题空间；
4. 证明作为构成原问题最优解的组成部分，每个子问题的解就是它本身的最优解。方法是反证法，考虑加入某个子问题的解不是其自身的最优解，那么就可以从原问题的解中用该子问题的最优解替换掉当前的非最优解，从而得到原问题的一个更优的解，从而与原问题最优解的



注意我们描述 乘法 的时候一般使用 $\times$ 或者 $\cdot$ ，特殊情况（如卷积）下会使用 $*$ （也可以写成 $\ast$ ）。标题是简洁的词组，但我们不希望正文部分由词组拼凑而成。上图中「两个要素」，建议更改为「动态规划的原理具有以下两个要素」，上下文保持连贯。可取的地方是，适当使用 有序 列表可以更有条理地表述内容。再次提醒，在使用列表的时候，每一项如果是一句话，需要在末位添加 标点符号。有序列表通常添加分号，在最后一项末位添加句号；无序列表统一添加句号。

例 5

看一张图就明白了



实现

树剖的实现分两个 DFS 的过程。伪代码如下：

第一个 DFS 记录每个结点的深度（deep）、子树大小（size）。

```
1 TREE-BUILD-DFS(u, dep)
2   u.deep=dep // 记录深度
3   u.size=1
4   for v is u's son
5     u.size+=TREE-BUILD-DFS(v, dep + 1)
6   return u.size // 返回该节点对应的子树的大小
```

第二个 DFS 记录每个结点的重子结点（heavy-son）、重边优先遍历时的 DFN 序、所在链的链顶

适当引用 图片 可以增强文章易读性。使用 伪代码 的方式表达算法过程可以方便又简洁地描述算法过程，相比于直接贴模板代码更加好懂。

树哈希

Method I

Formula

$$f_{now} = size_{now} \times \sum f_{son_{now,i}} \times seed^{i-1}$$

Notes ¶

其中 f_x 为以节点 x 为根的子树对应的哈希值。特殊地，我们令叶子节点的哈希值为 1。

$size_x$ 表示以节点 x 为根的子树大小。

$son_{x,i}$ 表示 x 所有子节点以 f 作为关键字排序后排名第 i 的儿子。

$seed$ 为选定的一个合适的种子（最好是质数，对字符串 hash 有了解的人一定不陌生）

上述哈希过程中，可以适当取模避免溢出或加快运行速度。

Method II

Formula

$$f_{now} = \bigoplus f_{son_{now,i}} \times seed + size_{son_{now,i}}$$

Notes

其中 f_x 为以节点 x 为根的子树对应的哈希值。特殊地，我们令叶子节点的哈希值为 1。

同样的问题，标题使用英文。并且在使用完括号后没有句号。另外，上图中的行间公式虽然没有使用艾弗森括号，但是由于下标嵌套过多，使得最底层的下标字体很小，整个公式也并不美观。建议将 $son_{\{now,i\}}$ 更换为 $son(now,i)$ ，或者把 $f_{\{now\}}$ 替换为 $f(now)$ 。我们希望尽量控制下标嵌套在两层以内（上标的运用主要是数学表达式，

因此可以允许多次嵌套，如，《上帝造题的七分钟》）。

例 7

LCT 通过 $\text{Splay}(x, y)$ 操作，将树上两节点 x 到根 y 的路径旋转成以 y 为根的 Splay 树，路径信息的修改和统计转化为平衡树上的操作，这使得 LCT 在维护树链信息上具有优势。此外，借助 LCT 实现的在树链上二比树链剖分少一个 $O(\log_2 n)$ 的复杂度。

例题 [luogu P1501\[国家集训队\]Tree II](#)

给出一棵有 n 个结点的树，每个点的初始权值为 1。 q 次操作，每次操作均为以下四种之一：

1. $- u_1 v_1 u_2 v_2$: 将树上 u_1, v_1 两点之间的边删除，连接 u_2, v_2 两点，保证操作合法且连边后仍是一棵树。
2. $+ u v c$: 将树上 u, v 两点之间的路径上的点权都增加 c 。
3. $* u v c$: 将树上 u, v 两点之间的路径上的点权都乘以 c 。
4. $/ u v$: 输出树上 u, v 两点之间的路径上的点权之和对 51061 取模后的值。

$1 \leq n, q \leq 10^5, 0 \leq c \leq 10^4$

- 操作可以直接 `Cut(u1,v1), Link(u2,v2)`

对树上 u, v 两点之间的路径进行修改时，先 `Split(u,v)`。

此题要求进行在辅助树上的子树加，子树乘，子树求和操作，所以我们除了一般 LCT 需要维护的子树翻转标记，还要维护子树加法标记和子树乘法标记。处理标记的方法和在 `Splay` 上是一样的。

在打上和上传加法标记时，子树权值的变化量和子树中的结点数有关，所以我们还要维护子树的大小 `siz`。

在下传标记时，需要注意顺序，先下传乘法标记再下传加法标记。子树翻转和子树加乘两种标记没有冲突。



参考代码

使用 MkDocs 扩展语法，让例题题面与算法描述区分开。将代码折叠，可以让文章更紧凑。（毕竟看 Wiki 的大多数是了解思路，除了模板代码需要阅读外，习题的代码大多可以折叠。）在描述函数操作时，使用行内代码和 LaTeX 公式都是不错的选择。

「NOI2001」食物链

「NOI2002」银河英雄传说

其他应用

最小生成树算法中的 Kruskal 是基于并查集的算法。

References

- [1]Tarjan, R. E., & Van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. Journal of the ACM (JACM), 31(2), 245-281.[ResearchGate PDF](#)
- [2]Yao, A. C. (1985). On the expected performance of path compression algorithms.[SIAM Journal on Computing](#), 14(1), 129-133.
- [3]知乎回答: 是否在并查集中真的有二分路径压缩优化?

🕒 本页面最近更新: 2019/7/24 02:06:49, [更新历史](#)

✎ 发现错误? 想一起完善? 在 [GitHub](#) 上编辑此页!

👤 本页面贡献者: [Ir1d](#), [TrisolarisHD](#), [stevebraveman](#), [HeRaNO](#), [JuicyMio](#), [Xeonacid](#), [sailordairy](#), [sshwy](#), [abc1763613206](#), [symkube](#), [Chrogeek](#)

© 本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用

评论

在文末罗列出参考文献, 可以使页面的内容更严谨, 真实可信。

外部链接

- [标点符号用法 \(GB/T 15834—2011\)](#)
- [维基百科: 格式手册/标点符号](#)
- [中文文案排版指北 \(简体中文版\)](#)
- [中文文案风格指南 - PDFE GUIDELINE](#)
- [一份 \(不太\) 简短的 LATEX2_ε 介绍或 106 分钟了解 LATEX2_ε](#)
- [中文出版物夹用英文的编辑规范](#)

参考资料与注释

- (冒号) 表示总结上文。↩
- 科学技术名称的英文全称与其缩略形式间, 应使用英文逗号。中文句子内夹用了用以注释、补充或说明的英文句子或语段, 该英文句子或语段用中文圆括号标示。↩
- [cstd::std::h namespace](#) ↩
- [CCF 关于恢复 NOIP 竞赛的公告 - 中国计算机学会](#) ↩
- [我的公式为什么在目录里没有正常显示? 好像双倍了](#) ↩
- [SVGIMDN](#) ↩
- [Save Page in Internet Archive](#) ↩
- [APNG](#) ↩
- [OI-wiki/OI-wiki#3422](#) ↩

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? 在 [GitHub](#) 上编辑此页!

本页面贡献者：[CoelacanthusHex](#), [StudyingFather](#), [billchenchina](#), [H-J-Granger](#), [NachtgeistW](#), [ouuan](#), [abc1763613206](#), [Alisahhh](#), [Back1ght](#), [BCFeather](#), [CCXXXI](#), [Chrogeek](#), [countercurrent-time](#), [diauweb](#), [Enter-tainer](#), [gi-b716](#), [Great-designer](#), [greyqz](#), [gtn1024](#), [hensier](#), [hsfzLZH1](#), [ImpleLee](#), [lr1d](#), [jifbt](#), [kenlig](#), [kLiHz](#), [ksyx](#), [megakite](#), [mgt](#), [ouuan](#), [shuzhouliu](#), [sshwy](#), [SukkaW](#), [Suyun514](#), [Tiphereth-A](#), [Xeonacid](#), [yuhuoji](#), [zwuis](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用